aegisbank参考一

🔗 原文链接： https://p6-flow-sign.byteimg.com/to...

⏰ 剪存时间：2026-10-18 22:30:47 (UTC+8)

✂️ 本文档由 飞书剪存 一键生成

AegisBank项目开发参考手册

——基于成熟Agent框架设计经验的技术落地指引

1 文档说明

本手册面向AegisBank安全自治银行AI智能体的启动开发阶段，从同赛道已落地的通用Agent框架项目中，提炼经过验证的可信Agent设计思路，为我们的技术架构、权限系统、功能优先级划分提供可直接落地的参考。本手册不涉及答辩演讲技巧，所有内容均指向开发过程中需要明确的模块设计、技术选型边界和功能拆解。

核心参考来源为同赛道项目「个人数据主权驱动的新型Agent协作框架」的完整技术宣讲，该项目在Agent身份信任、权限网关、工具标准化方面已经跑通了最小可用原型，其踩过的架构坑和总结的设计原则可以直接规避我们初期的设计弯路。所有参考内容均结合银行场景的合规要求做了适配调整，不直接照搬对方的去中心化技术路线。

2 参考项目核心设计思路复盘

2.1 Agent身份体系的四个必选属性

该项目提出，一个可被信任的数字Agent身份必须同时满足四个属性：唯一可识别、可验证身份、可被主动调用、操作可签名不可抵赖。当前行业里所有的Agent身份体系，要么只有账号ID没有签名能力，要么被锁在单家平台内部无法跨系统验证，这也是用户不敢把高风险操作交给Agent的核心原因之一。

2.2 网关是可信Agent的核心组件

该项目在开发过程中总结出：Agent不是一个聊天窗口，它本质是一个带权限闸门的服务网关——所有用户数据放在网关后面，所有外部调用必须经过网关的身份校验和权限判断，不存在绕过网关直接操作数据的路径。这和我们银行场景里的风控网关逻辑完全一致，只是该项目把它从服务端下沉到了每个个人Agent上。

2.3 工具标准化是Agent可扩展的前提

该项目踩过的最大架构坑是：初期把所有工具逻辑硬编码在Agent主流程里，后续每加一个功能就要改核心调度逻辑，发版成本极高。后来他们把所有工具都封装为带权限声明的标准API节点，新增工具只需要注册节点、声明需要的权限等级，Agent主流程不需要做任何修改，扩展成本直接降低一个数量级。

2.4 现有Agent项目的架构共性问题

该项目复盘同类失败项目时总结了三个共性架构缺陷，我们在开发初期就要主动规避：一是把Agent能力完全绑定在第三方聊天窗口里，没有独立的交互层；二是没有独立的数据层，所有用户数据都存在第三方平台，Agent没有数据控制权；三是工具系统不可插拔，功能扩展依赖主版本发版。

3 核心模块开发设计参考

3.1 操作签名与审计模块

开发优先级：P0。参考对方提出的「操作不可抵赖」原则，从项目初期就不要把日志做成事后审计的附属功能，而是作为整个Agent执行链路的核心组件。所有被Agent调用的工具节点，执行完成后必须生成带用户授权凭证、节点调用参数、执行结果的结构化签名日志，日志写入后不可篡改，任何一笔操作都可以反向追溯到原始用户指令。具体开发要求：日志必须包含触发时间、用户原始指令、DAG节点路径、工具调用入参、执行结果、用户确认状态六个字段，红色高危操作必须留存用户二次确认的签名凭证。

3.2 权限网关模块

开发优先级：P0。不要把权限判断散落在各个工具节点内部，必须做统一的前置权限网关，所有Agent的工具调用请求在进入DAG调度之前，先经过网关做三层校验：第一层判断当前工具对应的风险等级（绿/黄/红），第二层判断当前用户对该类操作是否已经完成授权，第三层判断当前请求是否存在越权、Prompt注入的风险。网关拦截后的处理逻辑要统一：绿色操作直接放行，黄色操作生成待确认请求推送给用户，红色操作要求用户完成二次身份校验。参考对方的设计，网关必须提供可查询的授权记录接口，前端可以随时查询某个用户给Agent开放了哪些权限，支持一键撤回。

3.3 标准化工具SDK模块

开发优先级：P1。从第一版开始就要避免把工具逻辑写死在主流程里，所有金融工具（查账单、查余额、转定期、卡片管理、订阅查询）都封装为独立的标准节点，每个节点注册时必须声明三个信息：节点的风险等级（绿/黄/红）、节点执行需要的最小权限、节点的入参出参结构。DAG调度器只负责按任务规划调用节点，不关心节点内部的业务逻辑。这样后续新增金融工具的时候，只需要按照SDK规范写一个独立插件，不需要改动核心调度代码。

3.4 交互授权流程模块

开发优先级：P0。参考对方提出的「不要用复杂OAuth流程增加用户负担」的原则，设计三级对应的交互流程：绿色操作直接静默执行，不打扰用户；黄色操作执行前10秒给用户发一个轻量确认通知，用户不操作就自动取消；红色操作必须暂停DAG流程，等用户完成明确的授权确认后再继续执行。不要把所有风险操作都弹一个很重的确认窗口，用户会逐渐养成盲目的确认习惯，反而失去风险拦截的意义。

4 功能对标清单

结合该项目点出的现有银行Agent架构缺陷，我们的第一版功能需要覆盖以下差异化能力，避免和市面已有的银行AI助手做成同质化产品：

现有银行AI助手的架构缺陷	AegisBank第一版必须实现的能力

所有能力锁在银行APP内置聊天框内，Agent没有独立的任务调度层	实现独立的DAG任务调度引擎，不绑定特定前端入口，支持多渠道唤起Agent完成任务
所有操作在平台侧黑盒执行，用户看不到Agent做了什么，也无法收回授权	前端提供操作记录查询页，用户可以查看Agent的全量执行日志，支持随时收回任意工具的调用权限
工具逻辑硬编码在主程序里，每加一个功能就要发版	实现标准工具插件SDK，每个工具节点自带风险等级声明，新增工具不需要改动核心调度逻辑
所有操作要么完全不让Agent做，要么直接放行，没有分级授权	实现完整的绿/黄/红三级风险拦截链路，不同风险等级对应不同的用户确认强度

数据来源：结合参考项目的架构经验与银行场景需求整理

5 MVP开发优先级

结合参考项目的最小可用原型经验，我们的开发不要一开始就追求全功能，按以下三个阶段迭代，每个阶段完成后都可以跑通完整的闭环：

5.1 第一阶段（最小可用闭环）

目标：跑通单银行场景下的绿/黄/红三级操作全链路。必须包含的模块：基础大模型意图解析、DAG任务调度器、统一权限网关、三个典型工具节点（查账单=绿色、调整还款日=黄色、小额转账=红色）、基础操作日志、简单的前端确认弹窗。这个阶段不追求多账户、跨场景能力，核心验证信任机制是否跑通。

5.2 第二阶段（核心能力完善）

目标：把信任机制做扎实。补充模块：Prompt注入与越权攻击的网关拦截、不可篡改的操作签名日志、用户侧权限控制台、更多高频金融工具节点（卡片管理、订阅管理、定期转账）、异常熔断机制。这个阶段重点把安全相关的能力做全，支撑银行场景的合规要求。

5.3 第三阶段（扩展能力）

目标：实现可插拔的工具生态。补充模块：标准工具SDK文档、插件化工具注册机制、多银行账户对接能力、跨场景任务规划能力。这个阶段再开始考虑跨平台、多入口的扩展，不要在第一版就做分布式架构，避免过度设计。

6 技术选型避坑红线

以下边界在整个开发过程中必须严格遵守，不能为了对标参考项目的技术炫点就偏离银行场景的合规要求：

第一，不采用参考项目里的去中心化P2P组网方案，所有跨服务调用必须走银行合规的开放接口，不做端到端的直连网络。

第二，不做「用户本地存储全量金融数据」的设计，所有敏感交易数据仍然存储在银行合规存储介质中，我们的网关只是做权限判断和操作拦截，不是替代银行的数据存储层。

第三，不要一开始就引入复杂的分布式协议、去中心化身份组件，MVP阶段用最成熟的中心化权限方案实现绿黄红分级即可，后续再根据需求迭代架构，避免为了炫技过度设计。

7 开发任务拆解

负责模块	第一阶段具体开发任务
调度引擎组	完成DAG任务调度器核心逻辑、基础任务规划链路、工具节点注册框架
安全网关组	实现绿/黄/红三级风险拦截逻辑、统一权限校验入口、基础操作日志记录
工具组	完成查账单、调整还款日、小额转账三个标准工具节点的开发，按SDK规范声明风险等级
前端组	实现基础聊天交互页、黄色/红色操作确认弹窗、简单的操作记录查询页